CSC 1300 LAB 10

SPRING 2026

CHOOSE YOUR OWN PROJECT!

CHOICE: Work with a Partner, or not

You may complete this assignment by yourself, or you may choose to write this program with <u>one</u> partner and practice paired programming.

If you do, both students must write both names in the comment block at the top and you must upload the exact same source/header files. Note that you may not share your code with anyone other than the one partner you select.

Concepts

- Structures
- Arrays & Pointers
- Functions, Loops, Branching

Lab Policies - What you Can't DO/Use in this lab

If you do any of the following, you will receive a 0 on some or all of the assignment and possibly get charged with an academic integrity violation (this is very serious).

1. You may not use code or syntax in your lab assignment **that has not been taught** in CSC 1300 class **without referencing your source**. How do you reference your source? By providing a link/URL in a multi-line comment in the section of code where you use "non-1300" code. Otherwise, this will be considered plagiarism. It is safer to use course materials (slides & example programs) to help you create solutions to the assignments.

2. **For this assignment, you may not use arrays, vectors, functions, classes, structs, or pointers**.

3. You may not (ever) use the following in your programming & lab assignments in CSC 1300:
   - while(true) or while(1)
   - goto statements
   - recursion
   - exit()
   - ternary operators
   - Any syntax requiring a '?'
   - lambda functions
   - try-catch

4. You **may not use generative AI (genAI)** **<u>in any way</u>** for this lab assignment including ChatGPT, Claude, Copilot, etc. Not sure if something is GenAI? Ask a TA or instructor!

5. You may not share your code with anyone (other than your paired programming parter, TAs for 1300, or instructors).

Lab Report

<u>After</u> you have completed this assignment, fill out the lab report and submit your lab report in your zip file along with your source files. Name your screenshot **lab10ReportProof**.
Lab Report Link: https://tntech.co1.qualtrics.com/jfe/form/SV_6M4wwN1g5Ynv01w

What to Turn In

Compress/zip the **Lab10_username** folder and upload it to the ilearn assignment folder named **Lab 10**. It should contain the following files:

- **Lab10.cpp**
- **Lab10.h**
- **Lab10ReportProof**

Directions

We want to give you the freedom to choose how you want to practice most of what you have learned in this class with this lab assignment.

You must follow the rules in the DO / DON'T table, and we are going to give you ideas so that you understand the expected scope of the program you must create, but you may come up with your own idea as long as all the rules/criteria are followed.

First, we have the **Do/Don't list** and then after that you will see the **ideas** of what you can implement.

DO	DON'T
You must create a header file named **Lab10.h** which must have your #includes, namespace, structure declarations, function prototypes, and include guards.	Create a program that doesn't make any sense or doesn't have any purpose at all.
Your program must contain at least one Structure declaration.	Don't create any classes.
You must create functions in addition to the main function to modularize your program. The number of functions needed depends on your program. All your function definitions should be in **Lab10.cpp**	Don't use library from the Standard Template Library (map, queue, stack, unordered_map, vector) or the algorithm library.
In your program, you must **dynamically allocate** an **array** of **structure variables** and the **size must be determined by the user**.	Don't create any non-constant global variables.
Your program must be implemented as a menu-based program where data must be able to be added, manipulated in some way (edit/modify/delete), and displayed/filtered in some way.	Don't use variable or function names that are nondescriptive.
Your program must contain a comment block at the top indicating your name, your partner's name (if you have one), date, filename, and purpose of the program.	Don't place your programmer-defined functions above your main function in Lab10.cpp
A comment block should be above each function explaining the name of the function and the purpose of the function.	Don't use generative AI of any kind (GitHub Copilot, ChatGPT, etc.)
Do use good programming practices and make your output readable (does the user know what to do?) as well as your code.	Don't share your code with anyone other than your paired programming partner.
Use consistent indentions and use white space well in your code and in your output. Make sure to compile your code and ensure there are no compile or logic errors. Test your program multiple times with various inputs. Make sure all numerical inputs are validated.	Don't turn in code that hasn't been compiled or tested thoroughly.

Program Ideas

1. **Dragon Rescue Registry**
   a. Dragon Structure: dragon name, type of dragon (fire, ice, earth, etc.), dangerLevel (1-10 scale), is it rescued? (Boolean)
   b. Menu: add a dragon, view all dragons, rescue a dragon, show dangerous dragons, show most & least dangerous dragon, quit
2. **Food Truck Frenzy**
   a. FoodTruck Structure: item name, price, calories, is it sold out? (Boolean)
   b. Menu: add menu item, mark item as sold out, display menu, show cheapest item
3. **Wizard Spell Book**
   a. Spells Structure: spell name, element (fire, water, lightning, etc.), power level, is the spell learned? (bool)
   b. Menu: learn a spell, cast a spell (mark as used/learned), show powerful spells, show most & least powerful spells
4. **Movie Night Planner**
   a. Movie Structure: movie title, genre, rating (1-10 or rotten tomatoes score), is it watched (bool)
   b. Menu: add a movie, mark as watched, show unwatched movies, recommend highest-rated unwatched
5. **Fantasy Sports Team Manager**
   a. Player Structure: player name, position, skill rating, is player injured? (bool)
   b. Menu: add player, bench injured players, show top players, show top players not benched, count active players
6. **Detective Case Files**
   a. Suspect Structure: suspect name, alibi, suspicion level, is cleared? (bool)
   b. Menu: add a suspect, clear a suspect, show most suspicious, list active suspects
7. **Space Mission Control**
   a. Astronaut Structure: astronaut name, mission name, oxygen level, is the astronaut safe? (bool)
   b. Menu: add an astronaut, update oxygen, flag danger (oxygen < 30%), show critical astronauts
8. **Grocery List Manager**
   a. GroceryItem Structure: item name, category (produce, meat, etc.), quantity, price, need to reorder?(bool)
   b. Menu: add item, mark purchased, show remaining items, count total items left, check stock & reorder
9. **Video Game Inventory System**
   a. Inventory Structure: item name, type (weapon, potion, armor), value, is the item equipped? (bool)
   b. Menu: add item, equip item, show equipped items, find most valuable item & least valuable item
10. **Concert Ticket Tracker**
    a. Ticket Structure: event name, seat number, price, is sold? (bool)
    b. Menu: add ticket, sell ticket, show available tickets, calculate total revenue.
11. **YOUR CHOICE!!!**
    a. Structure with at least 4 members and have at least one string, at least one numerical (int / double), and a bool
    b. Menu: (whatever makes sense for your data)